Serial Protocol Decode — measured reference

Ian Ameline · version 0.9 — beta · MIT licence

The detail behind MANUAL.md. Everything here was measured on a DMM6500 (firmware 1.7.17a) against an SDG2122X generator, verified where noted with an SDS1204X-E scope. The manual is written for someone using the app; this file is for someone who needs the numbers or is changing the code.

Press latency

A front-panel **touch** press is **not dispatched while a script is running**, and presses **queue** rather than being dropped. So a handler's own duration *is* its press latency, and returning is the only way to answer a touch press. That is why a stop press is absorbed after the capture it interrupted.

The **front-panel TRIGGER key was meant to be the exception** — latched by the trigger hardware rather than delivered to the interpreter, so a running handler could see it. Measured 2026-08-18, it is not: the press does not reach the latch while a panel-initiated run executes, so **nothing makes a minutes-long press interruptible**, and what makes it acceptable is only that the run is bounded and says its cost beforehand. See **Cancelling a run: the TRIGGER key latch** below.

Measured over 50 button presses covering every control in every state:

Press	Measured
Anything that does not capture	**77 ms or less** (Save is the slowest of them)
Capture, rate locked	**1.7 s**
Capture, unlocked, 9600 Bd and up	3.6 – 6.5 s
Capture, unlocked, 2400 Bd	4.9 – **9.1 s**
A recording, start to filed	**one press**, as long as the window takes — see below

The unlocked figure is a **range, not an average, and it is genuinely variable**: twelve back-to-back unlocked captures on one unchanged 2400 Bd line measured min 4.891 s, mean 5.358 s, max 7.253 s — a 2.36 s spread, 33 % of the maximum — and a separate run has been seen at 9.086 s. What varies is how many looks the rate probe needs, which depends on where the traffic is when the capture starts. `bench_panel.py` therefore bounds capture presses at **12 s**: margin sized to the measured spread rather than to a single sample, because a gate that fails at random teaches you to ignore it.

An unlocked capture costs more the **slower** the line, which is the opposite of the intuition. The rate probe starts at 1 MS/s, where 20 000 samples is 20 ms — less than one frame below about 1200 Bd — so it steps its sample rate down until it catches traffic, and each step is another capture. Locking removes the probe entirely, which is why the first press auto-locks.

That a capture exceeds 500 ms is deliberate. A 240-byte window is 19 200 samples and reading one sample out of the capture buffer costs 21.7 µs, so the read alone is 0.42 s before any decoding happens. Splitting one capture across several presses would shorten the press, but one press for one capture is worth more.

A recording press is minutes, not milliseconds, and that is now the design rather than a compromise. One press records a window, decodes all of it and files it: at 9600 baud that is 34 s of recording plus ~110 s of decoding for 32 kB, or 8.5 s plus ~30 s for 8 kB. Decoding runs at ~300 byte/s end to end and does not scale with the baud rate. `bench_panel.py` bounds such a press at **300 s**, which is a hang detector rather than a latency budget — the duration is the window size the operator chose, and TRIGGER ends it within about a second.

Sample rates and samples per bit

FRAME mode and the streaming modes pick their sample rate differently, and it matters:

- **FRAME** uses `fs_for_baud`, which oversamples from a discrete rate ladder, so the ratio is not constant: measured 8.33 samples/bit from 300 Bd to 19200 Bd, 8.68 at 115200 Bd, and 4.00 at 250000 Bd.
- **Streaming** uses `fs_for_burst`, the *lowest* listed rate that still delivers `minsabit` = 4 samples/bit, so it asks for ~4.17 samples/bit from 300 Bd to 19200 Bd and buys twice the recording length for it. Also ladder-dependent: 5.21 at 38400 Bd and 57600 Bd, 5.56 at 115200 Bd. The *delivered* ratio is slightly lower than the requested one, because `acq_overhead` costs ~0.51 us a sample.

Tolerance follows **samples per bit, not baud rate**, and the sample-rate ladder does not give every rate the same figure — 8.3 at 9600, 8.6 at 28800, 10.0 at 1500, 4.0 at 250 000. It holds at least ~8 up to 115200 Bd. Read `SA/BIT` rather than assuming a slow line is the robust one.

Recording length, and where the limits come from

There are **two recording modes**, capped at **8 192** and **32 768 bytes**. Nothing records without a byte ceiling; *Why there is no continuous mode* below gives the arithmetic, and beyond the ceiling the answer is flow control rather than a bigger buffer.

The two differ only in size, at every rate, and the choice is a responsiveness trade rather than a capability: one press records a window, decodes it and files it, so `8 kB` reaches bytes in about a quarter of the time and loses a quarter as much if cancelled, while `32 kB` spends less of the run on per-window overhead. Both are lossless.

The capture buffer holds **2 800 000 readings** (`ck_bufmax`). A mode asks for whatever its ceiling needs at the locked rate – `cap x framebits x (fs / baud)` – capped by the buffer. The table below is the 32 kB window; divide the readings and the durations by four for `8 kB`. Divide by `fs_for_burst` for the signal duration:

Locked rate	Readings	Signal	Wall clock to fill
300 Bd	1 365 334	1092 s	1092 s
1200 Bd	1 365 334	273 s	273 s
2400 Bd	1 365 334	136 s	136 s
4800 Bd	1 365 334	68 s	68 s
9600 Bd	1 365 334	34.1 s	34.1 s
19200 Bd	1 365 334	17.1 s	17.1 s
38400 Bd	1 706 667	8.5 s	17.1 s
57600 Bd	1 706 667	5.7 s	17.1 s
115200 Bd	1 820 445	2.8 s	18.2 s
160000 Bd	2 048 000	2.0 s	20.5 s

Signal and wall clock diverge above ~19200 Bd for the reason in the next section. Verified against the app's own arithmetic, not hand-computed.

The highest standard rate that records is 160000 Bd; the true cut-off is wherever `fs_for_burst` first returns nil, a little above it (165000 Bd still passes, 200000 Bd does not). The refusal is by name: *"200000 Bd is too fast to stream – no sample rate delivers 4 samples/bit; use FRAME"*. Recording needs 4 samples/bit and the digitizer stops at 1 MS/s, so 200000 Bd and beyond – including the 250 kBd decode ceiling – are FRAME only.

Nothing is dropped inside a recording. One hardware acquisition with no script running, so there are no gaps to fall through. Measured at 9600 Bd: **1925 of 1925 bytes contiguous** against a non-repeating payload, at a byte rate matching the wire to 0.5 %. Re-verified at 115200 Bd with the 1024-byte non-repeating `v71` payload: 400 checked bytes were **one unbroken slice** of it. And a full 2 800 000-reading recording at 4800 Bd decoded **67 749 bytes against 67 200 expected** for 140 s of line – a 0.8 % match, with the ASCII gutter showing continuous text straight through the payload wrap.

Beyond one window: flow control, not a bigger buffer

A window is not a total. `Options > Rear BNC = FC Out` asserts `trigger.extout` once per armed capture – `stream_arm` (serial_app.tsp), `acq_free` and `acq_triggered` (serial_core.tsp), with probe reads suppressed via `nofc` so a level-learning probe does not spend a credit. One pulse means "send up to one window". While Lua is decoding nothing is armed, so a device that waits for the credit cannot transmit into a closed window, and the byte log appends across windows. Unlimited total, bounded throughput.

And no interaction per window. `record_run()` loops credit -> record -> decode -> file -> credit inside the one press, so the operator presses Capture once for a transmission of any length. What ends it is under **The flow-control loop's bound** below.

Measured electrically: one pulse after arming, **4.92 V peak, -0.96 V baseline, 9.4-9.8 us wide** – SDS1204X-E at 1 GSa/s, armed SINGLE on the rising edge, two trials. The width CANNOT BE SET on this firmware: `trigger.extout.pulsewidth` is nil on 1.7.17a, so `sdec.fc_pulse` (100 us) is never applied and what comes out is the firmware default of ~10 us. **A device waiting for a credit therefore needs an edge-triggered input, an interrupt or a latch** – 10 us is 2.5 bit times at 250 kBd, and a polling loop can miss it. NOT measured as a closed loop – the SDG2122X cannot be gated by the DMM's trigger output on this bench, so no device that actually obeys the credit has been tried. Sound by construction, untested end to end.

Why there is no continuous mode

Continuous decode-to-file requires drain >= fill. Drain is `1/ck_win_us` = **20 833 samples/s**; fill is `fs_for_burst(baud)`, roughly 4x baud. The duty cycle that leaves:

baud	fill delivered	duty	slack per 16-byte file row
2400	9 949/s	**0.478**	34.8 ms
4800	19 798/s	**0.950**	1.66 ms
9600	39 200/s	1.882	impossible

2400 Bd is therefore the honest continuous ceiling. 4800 Bd is arithmetically alive on 5 % margin, but that margin must cover ~30 `file.write()` calls a second to /usb1, bookkeeping, a repaint and a stop check, and the USB write cost is **unmeasured** – so it is not a number to put on a panel. A mode that only worked at 2400 Bd would be too slow to be worth having, so the app offers a byte ceiling plus flow control instead.

`strm_maxbaud` = 4800 comes from the same inequality using a read cost of 27.3 us, which counts the buffer read and edge detection but **not** `ua_run`, the pass that frames bytes. With the applicable `ck_win_us` = 48 us the inequality is baud < 5208, so 4800 sits inside its own derivation only because the rate ladder has no entry between it and 9600 – at 95 % duty. Nothing reads the constant at runtime.

The shape for a continuous mode, if one is ever wanted, is **one ring buffer in `FILL_CONTINUOUS`** rather than two alternating buffers. A Lua re-arm between two buffers cannot be gapless – `abort()` is asynchronous and setting `fillmode` clears the buffer – whereas a ring driven by a `BLOCK_MEASURE_DIGITIZE` with `COUNT_INFINITE` has no seam, and `acq_fillmode()` already sets that mode. Three behaviours it depends on are undocumented on this instrument and need measuring first: reading a buffer while the trigger model writes to it, cursor semantics across the wrap, and `buffer.saveappend()` on a buffer being filled. `notes/DESIGN-double-buffer.md` has the analysis and the experiments. ### The reading buffer accepts ~100 000 readings/s, and that is wall clock, not signal

The durations above are **signal** time — buffer size ÷ the sample rate the app asks for — and they are correct. What they are not is how long you wait. Measured fill rates, polling `buf.n` from the host while a press-driven recording ran:

Baud	Mode	Asked	Achieved	Ratio
4800	32 kB	20 kS/s	19 798/s	0.99×
9600	32 kB	40 kS/s	39 200/s	0.98×
19200	32 kB	80 kS/s	76 858/s	0.96×
38400	32 kB	200 kS/s	99 996/s	0.50×
57600	32 kB	300 kS/s	99 986/s	0.33×
115200	32 kB	640 kS/s	99 994/s	0.16×
160000	32 kB	1 MS/s	100 006/s	0.10×

This is not a configuration mistake and there is nothing to fix. It was measured three ways at `samplerate` = 1 MS/s, and all three land on the same ceiling:

Configuration	Readings	Time	Rate
`digitize.count = 1`, `SimpleLoop(200000)` — what the app does	200 000	2.017 s	99 146/s
`BLOCK_MEASURE_DIGITIZE` with count = 200 000	200 000	2.119 s	94 400/s
blocking `dmm.digitize.read(buf)`, `count` = 200 000	200 000	2.186 s	91 502/s
blocking `dmm.digitize.read(buf)`, `count` = 1 000 000	1 000 000	10.930 s	91 487/s

Two things worth knowing from that table. **`SimpleLoop` takes exactly one reading per iteration and ignores `dmm.digitize.count`** — `count = 200000` with one iteration returns `n = 1`, so the loop count *is* the reading count on that path. And the blocking read, which is what FRAME mode uses and which genuinely samples at 1 MS/s, files readings at the same ~91 k/s. So the ceiling is the buffer write, not the digitizer and not the trigger model.

The digitizer really does sample at the rate it is told. At 115200 Bd the app asked for 640 kS/s and `acq_measure_fs` read back **479 677 S/s** with 4.16 samples/bit, and the bytes were contiguous and correct. The consequence is only wall clock: 505 723 readings gathered over 5.0 s of wall time represented **1.054 s of signal**, and the 12 171 bytes decoded match 1.054 s × 11 520 B/s to 0.2 %.

So above ~19200 Bd, filling the buffer takes roughly `capacity / 100 000` seconds regardless of rate — about 17-20 s for every mode in the table.

Cancelling a run: the TRIGGER key latch, and why it does not reach the operator

> **Status, 2026-08-18: this mechanism works and cannot be triggered by a finger.** The plumbing below is real — when the cancel event is delivered by a firmware trigger timer blended in as stimulus 2, a run stops within one poll, keeps every byte decoded so far and files them (measured: fired 8 s into an 8 kB run, `ck_cancel` true, `stopped` = `stopped`, 497 bytes kept, ended at 8.1 s against 20.5 s uninterrupted). What does **not** happen is the key reaching it: pressed 20 % of the way through a 32 kB decode, the run finished `full` and the latch was **empty** afterwards. The polls do run — counters over a 20.5 s run show `ck_progress` called 15 times and `cancel_asked` 21 times.
>
> The distinguishing detail, and the lead to test: the table below was gathered with `bench_cancelkey.py` driving a **host-initiated** script, whereas every failure is a **panel-initiated** run — the firmware executing a display button's event string. Presses may be barred from generating trigger events inside a display event handler. The reference is consistent with a state dependence: the key's action "depends on the instrument state" (3272), and running a script selects the trigger-model measurement method, where the key means Initiate/Abort Trigger Model (8060–8150).
>
> So the panel promises no stop, and the manual says a recording runs to its own end.

The problem. A decode long enough to need cancelling is a handler that has not returned, and a touch press is not delivered until it does. So the app cannot notice a Cancel button, and for a while the answer was to make the press *short* instead — one slice per press, which cost **twelve presses** to decode a full 32 kB recording.

What the latch does. The front-panel TRIGGER key generates `trigger.EVENT_DISPLAY`, and an event blender is a latching detector for it: "if one or more trigger events were detected since the last time `trigger.blender[N].wait()` or `.clear()` was called, this function returns immediately" (ref 14-334). The latch lives in firmware, so the interpreter being busy is irrelevant. Measured on 1.7.17a by `tools/bench_cancelkey.py`, **against a host-initiated script** — which is the caveat above:

Question	Measured
Does a press latch with **nothing armed**?	**Yes** — latched 2.9 s after the press, no waiting trigger model needed
Does the latch survive a **busy interpreter**?	**Yes** — pressed during a deliberate 10 s Lua spin, seen by the poll after it. Twice
Does `wait(t)` block, like `display.waitevent`?	**No** — returned in 1.1 ms at `t` = 0.001, 50.1 ms at `t` = 0.05
What does a poll cost?	**1.06 ms**, amortised over 200 polls
Does it disturb anything?	**No** — 0 event-log entries across every run

The app uses **blender 2**, because `acq_triggered()` owns blender 1 for the trigger-source OR. The poll timeout is **1 ms and never zero**: a zero timeout is exactly how `display.waitevent` wedges this instrument, and one API behaving that way is reason enough not to hand a zero to another.

Cancel latency is one decode window. The poll sits in the progress hook, which runs once per window: 20 000 samples × 48 µs = **0.96 s** worst case during a decode, and 0.5 s during a recording, where the acquisition loop polls twice a second. A press is never lost in between, because the latch holds it until read.

A press before the run must not cancel it, so every run calls `.clear()` first — the press that stopped the *last* run is still sitting in the latch, unread, because nothing polls it while the panel is idle.

A cancel during the recording stops the recording, not the bytes. It ends the acquisition and goes on to decode what was captured, which is what the old Capture-to-stop press did; a second press then stops the decode too.

Two mechanisms that do not work, and one that does for the harness

- **`*TRG` (`trigger.EVENT_COMMAND`) cannot cancel a running handler.** Measured: sent 2 s into a 6 s busy loop, it never reached the latch, because the command queue is only drained once the interpreter is idle. It is therefore deliberately *not* wired as a second stimulus — a stimulus that can only fire between runs, where `.clear()` discards it, would read as coverage and be none.
- **`display.waitevent()`** wedges the instrument; see below.
- **A trigger timer does fire mid-script**, because it is firmware: measured firing **2.019 s into a 6 s busy loop**. `bench_panel.py` wires `trigger.EVENT_TIMER1` as a temporary second stimulus so the sweep can test a cancel with no finger on the panel, and removes it afterwards.

What the cancel key changed elsewhere

Setting	Was	Now	Why
`strm_press`	`true` — a press per slice	`false` — one press does the job	a long press is stoppable now
`strm_maxsec`	20 s	1200 s	20 s was a wait an operator would tolerate without a stop control; it silently truncated every recording slower than 9600 baud. 1200 s is where `stream_maxwait` already caps, so the two bounds agree and neither is a hidden cap — a full 32 kB window is 1092 s at 300 baud
Decode presses for a full buffer	455, then 17, then 12	**1**	the chunking survives only as the cancel granularity

The press-driven path is still in the app (`sdec.strm_press = true`) and still tested, because it is the fallback for a firmware whose blender cannot latch the key — there, chunking is the only way to stay stoppable.

The flow-control loop's bound

Under `FC Out`, one press credits the device, records a window, decodes it, files it and credits again, ending when a credited window comes back silent. A device that never stops talking would otherwise hold the panel indefinitely, so the loop has **two** backstops, and it is worth knowing why one is not enough:

bound	value	what it limits
`fc_maxwin`	32 windows	the BYTES — 1 MB at the 32 kB window size
`fc_maxsec`	1200 s (20 min)	the WALL CLOCK, decode included

A window count is not a time. Each window's acquisition is bounded independently at `min(stream_maxwait, strm_maxsec)`, and the decode costs more than the acquisition did — measured at 9600 baud, 34 s to fill a 32 kB window and about 109 s to decode it. So 32 windows is **76 minutes** there, and at 300 baud, where every window waits out the full 1200 s acquisition

ceiling, the count alone allowed **ten and a half hours**. A panel that comes back the following afternoon is a power cycle in practice.

So the clock is the bound that binds: 20 minutes is roughly **eight** windows at 9600 baud, not 32. That is a deliberate reduction in bytes per press, and it costs no DATA — the sender only transmits when credited, so it is still waiting where the run stopped. What it costs is two presses and a file boundary, described below.

Both are backstops, not schedules: TRIGGER ends a run within about a second. When either fires the note row names which one and gives the remedy, **from the first window onward** — a run stopped by the clock on window one is the normal case on a slow line.

The remedy is `Mode`, then `Capture`, and it continues in a NEW file. Not one press, and not the same file: every way out of a recording goes through `mode_exit()`, which sets `capmode` back to `frame` *and* nils `flog_path`. So `Capture` on its own takes a frame capture, and the bytes that follow land in the next numbered `bytesNNN.txt`. **The data is not lost** — the device is still waiting for a credit that has not gone out, so nothing was transmitted in the gap — but a bounded run is *fragmented across two files*, not truncated.

> **Open decision for a future version.** If a bounded run should continue into the same file, that means keeping `flog_path` across a `mode_exit()` that ends on a bound — which interacts with `NewLog`'s "closed *file* — the next capture starts a new file" note and with the per-capture eviction rules. That is a design change rather than a fix, so it is recorded here and not made.

They matter most on a firmware where the cancel does not work at all. If the event blender or `trigger.EVENT_DISPLAY` is missing, `cancel_ok` is false, `cancel_pressed()` answers false for ever, and these two are the only exits the loop has.

The idle watchdog and remembered levels

A recording's quiet-line exit needs a voltage threshold, and `clear_result()` zeroes the measured one at the top of every capture — so `stream_begin()` probes the line first. **Under flow control that probe looks at a silent line**, because the device is waiting for its credit, and the same is true of every window after the first. Without a fallback the watchdog would be disarmed exactly where it matters most, and every window would wait out `strm_maxsec`.

So `sig_levels()` keeps the last levels it successfully measured (`lvl_thr`, `lvl_hyst`, `lvl_swing`), past `clear_result()`, and `stream_begin()` falls back to them when its own probe finds nothing. They exist whenever a rate is locked, and locking a rate is already required to enter a recording mode. With no remembered levels the watchdog stays disarmed, which is the safe answer: a truncated recording is worse than a long one.

`display.waitevent(0)` blocks and wedges the instrument. Do not use it. The reference documents it as `<object id>, <sub id> = display.waitevent([timeout])`, and the instrument's own display example polls it with a 0 timeout inside a long loop to catch a Stop press – the shape that would let a long operation stay cancellable.

On firmware 1.7.17a a 0 timeout does not mean "return immediately with whatever is pending". With no waitevent-enabled object to report, it waits: the TSP interpreter stops answering, the control socket goes silent, and reconnecting does not recover it. Only a power cycle does. Confirmed both with the app running and on a bare instrument with nothing built. Recorded as `DMM_WAITEVENT_WEDGES_THE_INSTRUMENT` in `tools/instruments.py`.

A second obstacle sits behind it: an object's event slot is *either* a `waitevent` flag *or* a command string, never both (Display API reference, `setevent`), and every button in this app is a command string.

`display.OBJ_TIMER` is **not** a DMM6500 object type. It appears nowhere in this instrument's display API reference, so there is no timer object to drive work from.

Tolerance envelope

Measured at 8 samples per bit. In every case beyond the limit the failure is **reported** rather than silent, except where noted.

Condition	Byte-exact to
Clock error, rate **locked**	**±4 %** — past this, running *fast* corrupts bytes silently
Clock error, **auto-detect**	±12 % and beyond — it measures rather than assumes
Timing jitter	**±15 %** of a bit time — ±15.6 µs at 9600 Bd, ±1.3 µs at 115200. Past ±20 % it can fail silently
Amplitude noise	~40 % of the logic swing — ~1.3 V p-p on a 3.3 V line
Low-pass filtering (poor probe, long cable)	RC up to **0.7 bit times** — a ~2.2 kHz filter at 9600 Bd
Slow edges (open-drain pull-up)	rise up to ~40 % of a bit time — 42 µs at 9600 Bd, 3.5 µs at 115200
Impulse spikes / dropouts	usually costs bytes and raises errors; a spike confined to a *data* bit changes that byte undetectably
Logic swing	**1.6 V** to 5 V TTL — the limit is the swing, not the family
DC offset	anything keeping both levels inside ±10 V

The UART framing cliff itself is 0.5/9.5 = 5.26 %; `ratemargin` = 0.04 is where the app starts warning.

Levels and offsets, verified

Byte-exact from −5 V to +5 V of offset with a 3.3 V swing, and across the same offset range with a **1.6 V** swing — including 1.6 V sitting at 5.05…6.65 V and at −5.06…−3.46 V. A 60 mV swing is refused – by the swing floor (`minswing` = 0.10 V, which reports the measured swing) or as a line with no transitions, depending on which test trips first. The useful floor is between the two.

Rates verified on hardware

43 matrix points and 19 non-standard rates, byte-exact, with no instrument event logged: six frame formats, the standard ladder 300 – 250 000 Bd, a 1 kB non-repeating payload, three logic swings and twelve DC offsets.

Non-standard rates matter because a device with an awkward crystal divisor does not sit on the standard ladder: 900, 1500, 2800, 3600, 7200, 12000 and 16000 Bd all decode byte-exact, as do arbitrary values like 1379, 2731, 5333, 8123, 13333, 21600, 29127, 41666, 53333, 71111, 89123 and 104857.

A rate within **2 %** (`snaptol`) of a standard one is *reported* as the standard one — 29127 Bd reads as 28800 — because real devices use standard rates and 2 % is far inside the framing margin. The bytes are unaffected.

Above the 250 kBd ceiling it refuses by name rather than reporting a submultiple: 255 kBd, 260 kBd and 300 kBd were each declined with the samples-per-bit reason.

A known marginal at 2400 Bd

The 1 kB non-repeating payload at 2400 Bd (25 kS/s) has produced flagged bytes in every sweep run: 1 flagged in two runs, 2 bad interior frames in a third. **The app flags them — it has never decoded that point silently wrong.** Whether the cause is the generator's arb playback at that rate or the decoder is not yet established; the offsets need checking for repeatability. Tracked, not fixed.

Ambiguity, and what FIT does not tell you

`FIT` is not a probability that the rate is right. It measures how well one bit period explains the measured pulse widths. A clean fit can still be an octave out, which is why the note row names a rival rate when one fits.

Perfectly periodic traffic can be genuinely undecidable: eight `0x00` frames at 9600 Bd 7N1 with a one-bit gap are bit-for-bit the same waveform as four `0x08` bytes at 4800 Bd 8N1. No decoder can tell which device sent it, so the app says so instead of guessing. Irregular gaps between bytes remove the ambiguity entirely.

A capture beginning mid-byte costs **2 to 12 bytes**, measured across thirty baud rates, and does not scale with the rate — it is at most one frame plus the trigger's own offset. Those bytes are displayed but their bit boundaries are wrong, so they are arbitrary groupings of real wire bits.

Data Bits — why `Auto (any)` is not the default

`Auto (7/8)` searches 7 and 8 data bits. `Auto (any)` also tries 5, 6 and 9. Measured over 105 hostile-signal cases, the wider search changed the answer in **5** of them, and only **1** was a genuine 9-bit stream: the other 4 were damaged captures that a wider frame made look *cleaner* while being wrong, because the extra data cell absorbs whatever broke the stop bit. 9 data bits is searched **only** when explicitly forced, for the same reason.

Only one stop bit is searched and reported: a second stop bit is indistinguishable from a bit time of idle, which the decoder already tolerates. A 2-stop line decodes correctly and reports 1 stop.

Auto-lock conditions

Auto-lock is on by default and conditional. It locks only when **all** of these hold:

- the rate snapped to a standard one (within 2 %)
- at least 8 good frames, and a clean unbroken run covering ~60 % of the capture
- `FIT` at 0.9 or better
- the logic levels were stable — no drifting or noisy baseline

Otherwise the padlock stays amber and **Lock Rate** is offered. Polarity is **never** locked; it is re-derived every capture, because freezing it off one short detect capture is how a confidently inverted, entirely wrong decode happens.

Flow control

`Options ▸ Rear BNC = FC Out` makes the rear EXT TRIG OUT emit **one** ~5 V, ~10 µs pulse once the digitizer is armed — 4.92 V and 9.4–9.8 µs on a scope. The width is the firmware's own and cannot be set here, so the receiving device needs an edge-triggered input rather than a polling loop. It is a **credit, not CTS**: treat the pulse as permission to send no more than one capture can hold. It does nothing unless the device is built to wait for it.

One press then runs the whole conversation, one credit per window, until a credited window comes back silent or the 32-window bound fires. The device has to go quiet when it has nothing to send: that silence is the only signal the app has that the transmission is over.

Firmware limits worth knowing

One app build per power cycle. `sdec.start()` builds the display objects and the firmware does not appear to return the pool fully, so after enough rebuilds in one power cycle the instrument reports *"the maximum number of objects have already been created"* and the app refuses to build rather than coming up with controls missing. How many is enough is **not characterised** — it has been seen after a few dozen. One relaunch is safe; a long editing session needs a power cycle. This affects development, not normal use.

The build itself is **122 display objects** live after `ui_build()`, and **134** once the options screen has been built too. Counted with the harness census, not estimated.

Event 4915 — "attempting to store past the capacity of a reading buffer" — is ERROR severity, which the front panel shows as a **modal dialog over the app** whatever `localnode.showevents` says. Two defences apply to every armed capture: `fillmode = 1` per capture (the numeric value; `buffer.FILL_CONTINUOUS` does not exist on this firmware and assigning it raises), and 100 readings of capacity past the count the trigger model is asked for. Measured with no headroom: 20–30 events per capture.

`localnode.showevents` governs the remote interface only. It cannot suppress the panel's dialog, so muting is not a defence — not posting the event is.